

535514: Reinforcement Learning

Lecture 6 — Variance Function Approximation

Ping-Chun Hsieh

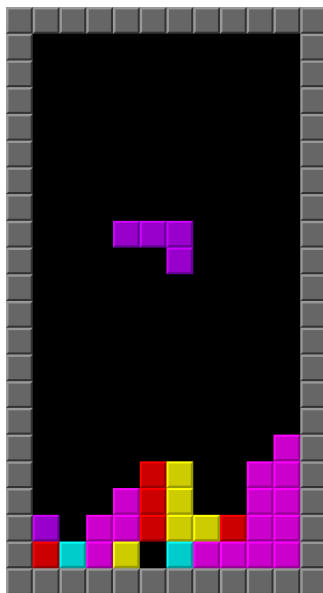
March 25, 2025

So far, we presume that $V^\pi(s)$ and $Q^\pi(s, a)$ can be predicted accurately via model-free prediction.

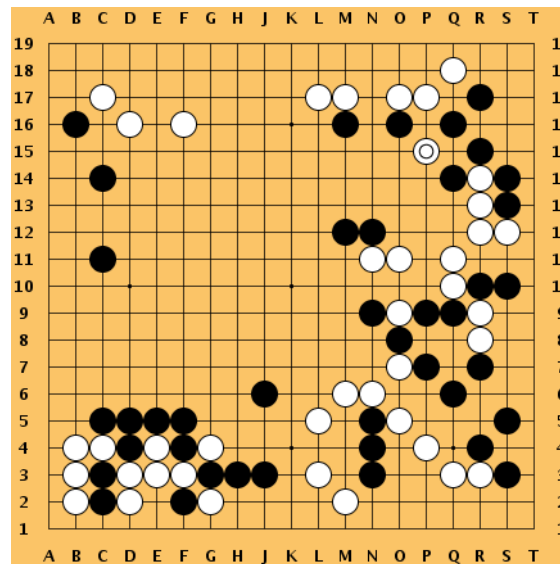
But, is this always true?

Tabular Methods for Large RL Problems?

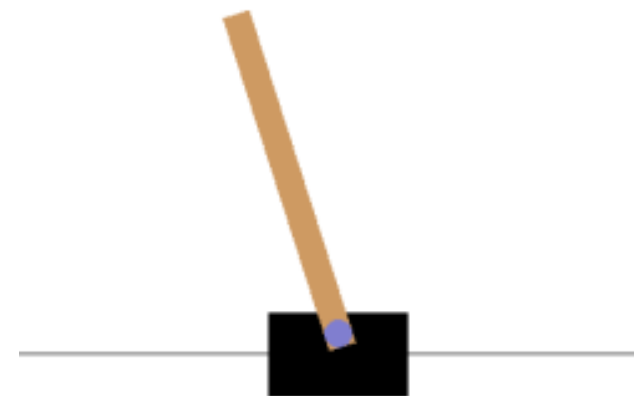
- ▶ So far: Value functions ($V(s)$ or $Q(s, a)$) are represented by a **lookup table**
- ▶ Such tabular methods may be impractical for large RL problems
 - ▶ Too many states (or actions) to store in memory
 - ▶ It is slow to learn $V(s)$ or $Q(s, a)$ for each state separately
- ▶ **Examples:**



Tetris: $\sim 2^{200}$ states



Go: $\sim 10^{170}_3$ states



Cartpole: continuous state space

Value Function Approximation (VFA)

- ▶ To scale up the model-free methods, **function approximation** is commonly used to learn value functions
- ▶ **Idea**: Approximate a value function by a parametric function

$$s \longrightarrow \boxed{\mathbf{w}} \longrightarrow \hat{V}(s; \mathbf{w}) \approx V^\pi(s)$$

$$\begin{array}{l} s \longrightarrow \\ a \longrightarrow \end{array} \boxed{\mathbf{w}} \longrightarrow \hat{Q}(s, a; \mathbf{w}) \approx Q^\pi(s, a)$$

$$\begin{array}{l} s \longrightarrow \end{array} \boxed{\mathbf{w}} \begin{array}{l} \longrightarrow \hat{Q}(s, a_1; \mathbf{w}) \\ \vdots \\ \longrightarrow \hat{Q}(s, a_{|\mathcal{A}|}; \mathbf{w}) \end{array} \quad \begin{array}{l} (\mathbf{w} \text{ updated} \\ \text{via MC or TD}) \end{array}$$

- ▶ **Motivation**: Generalize from seen states to unseen states

2 Commonly-Used Function Approximators

1. Linear combinations of features

2. Neural networks

- ▶ Both are differentiable function approximators

How to Quantify the Accuracy of VFA?

- ▶ For each state s , the squared error between $V^\pi(s)$ and $\hat{V}(s; \mathbf{w})$:

$$\left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right)^2$$

- ▶ To jointly consider all states, we use **mean squared error (MSE)**:

$$F(\mathbf{w}) := \sum_{s \in \mathcal{S}} \rho(s) \left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right)^2$$

- ▶ $\rho(s)$ are the weights for mixing the MSE of the states
- ▶ If $\rho(s)$ is a probability distribution, the objective function becomes:

$$F(\mathbf{w}) := \mathbb{E}_{s \sim \rho(s)} \left[\left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right)^2 \right]$$

Some Natural Choices of $\rho(s)$

- ▶ For **continuing** environments:

1. choose $\rho(s) \equiv d_{\mu}^{\pi}(s) := \mathbb{E}_{s_0 \sim \mu} \left[(1 - \gamma) \sum_{t=0}^{\infty} \gamma^t P(s_t = s \mid s_0, \pi) \right]$
2. choose $\rho(s) \equiv$ undiscounted stationary state distribution

- ▶ For **episodic** environments:

1. choose $\rho(s) \equiv d_{\mu}^{\pi}(s) := \frac{\mathbb{E}_{s_0 \sim \mu} \left[\sum_{t=0}^T \gamma^t P(s_t = s \mid s_0, \pi) \right]}{\text{normalization constant}}$

- ▶ **Remark:** $\rho(s)$ usually corresponds to the sampling strategy (will be discussed momentarily)

Value Function Approximation via GD

- **Goal**: find $\mathbf{w}$ that minimizes MSE between $V^\pi(s)$ and $\hat{V}(s; \mathbf{w})$

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \underbrace{\mathbb{E}_{s \sim \rho(s)} \left[\left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right)^2 \right]}_{=: F(\mathbf{w})}$$

- **Suppose**: We are given an oracle for querying $V^\pi(s)$
- Iterative GD update:

$$\begin{aligned} \mathbf{w}_{k+1} &= \mathbf{w}_k - \alpha_k \nabla_{\mathbf{w}} F(\mathbf{w}) \\ &= \mathbf{w}_k - \alpha_k \mathbb{E}_{s \sim \rho(s)} \left[\left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s; \mathbf{w}) \right] \end{aligned}$$

- GD finds a local minimum under proper step sizes α_k (Why?)

Value Function Approximation via SGD

- ▶ Iterative GD update:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \mathbb{E}_{s \sim \rho(s)} \left[\left(V^\pi(s) - \hat{V}(s; \mathbf{w}_k) \right) \nabla_{\mathbf{w}} \hat{V}(s; \mathbf{w}_k) \right]$$

- ▶ Iterative SGD update by the sampled state $S \sim \rho$:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \left[\left(V^\pi(S) - \hat{V}(S; \mathbf{w}_k) \right) \nabla_{\mathbf{w}} \hat{V}(S; \mathbf{w}_k) \right]$$

- ▶ The SGD update converges to a local minimum under stochastic approximation conditions of α_k (even for NN)

In practice, we don't have an oracle for $V^\pi(s)$.
What shall we do?

Monte-Carlo Value Function Approximation

- **Recall:** Iterative SGD update by the sampled state $S \sim \rho$:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \left[\left(V^\pi(S) - \hat{V}(S; \mathbf{w}_k) \right) \nabla_{\mathbf{w}} \hat{V}(S; \mathbf{w}_k) \right]$$

- **Idea:** Use MC, i.e. sample return G_t to estimate $V^\pi(S)$

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \left[\left(\textcolor{red}{G}_t - \hat{V}(S; \mathbf{w}_k) \right) \nabla_{\mathbf{w}} \hat{V}(S; \mathbf{w}_k) \right]$$

G_t is an unbiased estimate of $V^\pi(S)$

- Equivalent to *supervised learning* with (noisy) training data as

$$(s_0, G_0), (s_1, G_1), \dots, (s_t, G_t) \dots$$

Monte-Carlo Value Function Approximation (Cont.)

► First-Visit MC with Value Function Approximation:

Step 1: Initialize $\mathbf{w} = 0$ and $k = 1$

Step 2: Sample $\tau_k = (s_0, a_0, r_1, \dots, s_{L_k-1}, a_{L_k-1}, r_{L_k}) \sim P_{\mu}^{\pi_{\theta}}$

For each step of the current episode $t = 0, 1, \dots, L_k - 1$

If First visit to state s in episode k

$$G_t(s) = \sum_{i=t+1}^{L_k} r_i$$

$$\Delta \mathbf{w}_k \leftarrow \Delta \mathbf{w}_k + \alpha_k \left[\left(G_t(s) - \hat{V}(S; \mathbf{w}_k) \right) \nabla_{\mathbf{w}} \hat{V}(S; \mathbf{w}_k) \right]$$

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k + \Delta \mathbf{w}_k, \quad k \leftarrow k + 1$$

- Convergence to a local minimum can be achieved with proper step sizes α_k , for general function approximators (Why?)

Temporal-Difference Value Function Approximation

- **Recall**: Iterative SGD update by the sampled state $S \sim \rho$:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \left[\left(V^\pi(S) - \hat{V}(S; \mathbf{w}_k) \right) \nabla_{\mathbf{w}} \hat{V}(S; \mathbf{w}_k) \right]$$

- **Idea**: Use **bootstrapping** (e.g. TD(0)) to estimate $V^\pi(S)$

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \left[\left(r + \gamma \hat{V}(S'; \mathbf{w}_k) - \hat{V}(S; \mathbf{w}_k) \right) \nabla_{\mathbf{w}} \hat{V}(S; \mathbf{w}_k) \right]$$

$r + \gamma \hat{V}(S'; \mathbf{w}_k)$ is a biased estimate of $V^\pi(S)$

- Equivalent to *supervised learning* with (noisy) training data as

$$(s_0, r_1 + \gamma \hat{V}(s_1; \mathbf{w})), \dots, (s_t, r_{t+1} + \gamma \hat{V}(s_{t+1}; \mathbf{w})) \dots$$

- This can be easily extended to the more general TD(λ)

TD Value Function Approximation (Cont.)

- ▶ TD(0) with Value Function Approximation:

Step 1: Initialize $\mathbf{w} = 0$ and $k = 1$

Step 2: Sample $\tau_k = (s_0, a_0, r_1, \dots, s_{L_k-1}, a_{L_k-1}, r_{L_k}) \sim P_{\mu}^{\pi_{\theta}}$

For each step of the current episode $t = 0, 1, \dots, L_k - 1$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[\left(r_{t+1} + \gamma \hat{V}(s_{t+1}; \mathbf{w}) - \hat{V}(s_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s_t; \mathbf{w}) \right]$$